Energy Consumption Optimizer

IMPORTANT VIVA Q&A GUIDE

COURSE: MACHINE LEARNING LAB (SEM-2)

Case Study | Regression | Feature Scaling | Model Deployment

Focus: Conceptual questions on preprocessing, feature scaling, model selection, and deployment.

Prepared for Presentation Team:

- Gaurav Mandle (SCA25MCA098)
- Akash Raj (SCA25MCA080)
- Mithun Baadkar (SCA25MCA115)

Q1: What is the difference between StandardScaler and MinMaxScaler? When should you use which?

EXPLANATORY ANSWER:

StandardScaler normalizes data to have a mean of 0 and a standard deviation of 1. It is ideal for models that assume normally distributed inputs, such as Linear/Logistic Regression, Support Vector Machines, and Neural Networks.

MinMaxScaler compresses features to fit strictly between 0 and 1. It preserves 0 values and does not compress relative variances. It is preferred when features have rigid boundaries, or when algorithms do not assume standard distributions, such as K-Nearest Neighbors (KNN), Image Processing, or Neural Networks with sigmoid output layers.

Note: StandardScaler is generally less sensitive to outliers than MinMaxScaler, which squashes outliers and shrinks normal values into a very tight cluster.

Q2: Why is the R^2 score of your Linear Regression model so low (1.92%)? Does this mean the model failed?

EXPLANATORY ANSWER:

The low R^2 score of 1.92% indicates that a straight line is insufficient for explaining the variance of the electricity load. In our synthetic data, load changes cyclically over the 24-hour day (sine wave) and has a V-shape absolute relationship with temperature (more power is used for heating in cold weather and cooling in hot weather).

Since a Linear Regression model can only construct a straight line or flat hyperplane, it cannot model these waves and curves. The model did not fail programmatically - it was simply limited by its linear architecture. This highlights the necessity of using non-linear models for non-linear data patterns.

Q3: What is data leakage in feature scaling, and how did you prevent it? (Explain the Scaling Rule)

EXPLANATORY ANSWER:

Data leakage occurs when information from the test dataset is used to train or scale the model. During feature scaling, we must calculate the scaling parameters (mean and standard deviation for StandardScaler) using the training set ONLY.

We do this by running standard_scaler.fit_transform(X_train). We then apply these learned parameters to the test set using standard_scaler.transform(X_test) without recalculating them. If we fit on the entire dataset or fit separately on the test set, we leak test-set parameters into the training pipeline, yielding artificially high test scores that fail on real-world inputs.

The Scaling Rule is a strict reminder to always fit standard scalers on training datasets only, then apply transform to test/validation data.

Q4: How did you simulate the electricity load? What real-world patterns were introduced?

EXPLANATORY ANSWER:

We simulated 8,760 hours (24 hours x 365 days) of hourly data. The simulation incorporates:

1. A diurnal cycle: load = 300 + 100 * sin(pi * (hour - 6) / 12), creating peak demand in the afternoon and low demand at night.
2. A temperature effect: load increases by 2.5 kWh for every degree away from 22 degrees Celsius (hvac cooling/heating demand).
3. A weekend drop: load decreases by 30 kWh on Saturdays and Sundays to account for closed offices.
4. Sensor noise: random Gaussian noise (mean 0, std 20) to mimic real-world unpredictability.

Q5: Why do we scale features? What happens to Linear Regression coefficients if features are unscaled?

EXPLANATORY ANSWER:

If features are unscaled, their numerical magnitudes differ wildly (e.g. humidity is 40-90, is_weekend is 0-1).

In gradient-descent-based models, features with larger magnitudes dominate the updates, causing the optimization path to oscillate and take much longer to converge. In distance-based models (KNN, SVM), larger features dominate distance calculations entirely.

For closed-form Linear Regression, unscaled features result in coefficients that are difficult to interpret - a feature on a tiny scale will have an artificially huge coefficient to make an impact, whereas a feature on a large scale will have a tiny coefficient. Scaling brings all features to the same scale, allowing us to compare coefficient weights directly.

Q6: Why did the Random Forest model perform so much better (91.12% R^2)? How does it work?

EXPLANATORY ANSWER:

Random Forest is an ensemble of Decision Trees. Instead of attempting to fit a single global line through the entire dataset, a Decision Tree splits the feature space into small, localized regions (or boxes) and predicts the mean target value in each region.

By averaging predictions from 100 different decision trees (each trained on random subsets of data and features), Random Forest acts as a step-function estimator. It can approximate any non-linear curve (like our load's sine wave or temperature's absolute deviation) to high precision. This is why it achieved a 91.12% R^2 score compared to Linear Regression's 1.92%.

Q7: How do you handle missing values in a real-world dataset? What did you do here, and what are the alternatives?

EXPLANATORY ANSWER:

In this project, we used mean imputation (replacing missing temperature and humidity values with the average value of the respective column).

In a real-world dataset, alternatives include:

1. Dropping rows: only suitable if missingness is very low (< 5%) and random.
2. Median Imputation: preferred if the column has severe outliers.
3. Mode Imputation: used for categorical variables.
4. Forward/Backward Fill: preferred for sequential time-series data.
5. Model-based Imputation: using algorithms like KNN Imputer or Iterative Imputer (MICE) to predict missing values based on other features.

Q8: Explain the model deployment architecture. What is the role of scaler.pkl in production?

EXPLANATORY ANSWER:

In production, the model is serialized using joblib and saved as electricity_model.pkl, and the scaler parameters are saved as scaler.pkl. We wrap them in a REST API using Flask or FastAPI.

When a client dashboard sends a new prediction request (e.g., Temperature: 35C, Hour: 18), the API receives this raw JSON.
Because the model was trained on scaled features, the raw inputs MUST be scaled before running model.predict(). We load scaler.pkl to apply the exact training mean and scale: scaled_inputs = scaler.transform(raw_inputs). The scaled inputs are then fed into the model to generate the prediction.

Q9: How does Random Forest handle out-of-distribution or extrapolation (e.g. extremely high temperatures)?

EXPLANATORY ANSWER:

Random Forest cannot extrapolate beyond the range of its training data. Because a decision tree predicts the mean of the training samples in its leaf nodes, its predictions are bounded by the minimum and maximum target values present in the training set.

If temperature rises to 55C in production (higher than any training value), the tree will route the sample to the leaf node representing the highest trained temperature and predict that trained average. In contrast, Linear Regression will extrapolate infinitely along its fitted slope. In situations requiring extrapolation, linear models or deep neural networks with active slopes are preferred over tree-based models.

Q10: Explain the evaluation metrics: Mean Squared Error (MSE) and Root Mean Squared Error (RMSE).

EXPLANATORY ANSWER:

1. MSE (Mean Squared Error): Measures the average squared difference between predicted and actual values. Mathematically, it is 1/N * sum((y_actual - y_pred)^2). Because it squares the errors, it heavily penalizes large errors or outliers.
2. RMSE (Root Mean Squared Error): The square root of MSE. Taking the square root brings the metric back into the same units as the target variable (kWh of electricity load). This makes it highly interpretable for domain experts (e.g. 'our model is off by an average of 22.16 kWh').